

UFMA – Universidade Federal do Maranhão

Integrantes do Grupo: Millena Gomes Andrade de Menezes Braga, Cássio Herberth Rodrigues Araújo, Stefany Costa de Almeida, Arthur Felipe Mourão de Oliveira, Alana Mayara Silva Monteiro

Professor Orientador: Luiz Henrique Neves Rodrigues

Desenvolvimento do Jogo “Snake” em Linguagem Assembly (MIPS)

Relatório de Progresso: Implementação de I/O e Renderização Gráfica

Objetivo da Etapa: Nesta fase do projeto, a equipe concentrou-se na implementação da comunicação entre a UCP e os periféricos, utilizando a técnica de Entrada e Saída Mapeada em Memória (Memory-Mapped I/O). O foco foi estabelecer o controle visual do jogo através do Bitmap Display e a captura de comandos do jogador via teclado simulado no MARS versão 4.5.

1. Mapeamento e Atualização de Vídeo (Bitmap Display)

Para renderizar os gráficos do jogo, estamos aplicando na prática a manipulação direta de memória. Tratamos a tela como uma grade contígua na memória principal.

- **Definição do Endereço Base:** O Bitmap Display é configurado no MARS para monitorar um segmento de memória a partir de um endereço base (ex: o início da *Heap*, 0x10040000).
- **Renderização por Coordenadas:** Como a resolução sugerida é de 256x256 pixels, a equipe precisa converter coordenadas cartesianas (X, Y) em um endereço linear de memória.
- **Cálculo de Deslocamento (Offset):** O endereço exato para desenhar um segmento da cobra é calculado através da seguinte fórmula em linguagem de máquina:
$$\text{Endereço Alvo} = \text{Endereço Base} + ((Y * \text{Largura da Tela}) + X) * 4$$
- **Escrita na Tela:** Após o cálculo do endereço, utilizamos a instrução

SW(*Store Word*) para injetar o valor hexadecimal da cor desejada diretamente naquele espaço de memória, o que altera imediatamente o pixel no Bitmap Display.

2. Interação com o Teclado (Técnica de Polling)

Devido às limitações do simulador MARS, que não permite paralelismo real, a leitura assíncrona do teclado não é viável. Portanto, a equipe implementou a captura de teclas utilizando a técnica de *polling* para o teclado.

- **Verificação de Status (Polling):** Dentro do laço de repetição principal (Game Loop), o programa realiza checagens contínuas no Registrador de Controle do Receptor, mapeado no endereço de memória `0xFFFF0000`.
- **Deteção de Evento:** O código avalia o bit menos significativo (Bit 0) deste endereço. Se o valor for 1, indica que um dado de caractere está disponível para leitura.
- **Captura do Dado:** Apenas quando o evento é confirmado, o programa faz a leitura do Registrador de Dados do Receptor, localizado no endereço `0xFFFF0004`, capturando o código ASCII da tecla pressionada.
- **Atualização de Rota:** O código ASCII capturado é traduzido em uma direção lógica, que atualizará os parâmetros de movimentação no próximo ciclo do Game Loop.

3. Relação Arquitetural: Processador MIPS (Projeto) e o Modelo IAS (Von Neumann)

Para contextualizar a implementação do projeto, é fundamental estabelecer o paralelo entre a arquitetura MIPS (simulada via MARS) e o modelo pioneiro IAS, baseado na arquitetura de Von Neumann. Embora o MIPS seja uma arquitetura RISC moderna, seu funcionamento central herda e evolui os preceitos clássicos estabelecidos pelo IAS, cumprindo diretamente os requisitos do nosso TAP.

- O Conceito de Programa Armazenado (Memória Unificada): A semelhança mais profunda entre as arquiteturas é o uso do "programa armazenado". No modelo IAS, dados e instruções compartilham a mesma memória principal. O nosso projeto visa aplicar exatamente essa "manipulação direta de memória". Em nosso código-fonte no MARS, as diretivas `.text` (onde ficam as instruções do jogo) e `.data` (onde armazenamos variáveis como o tamanho da cobra ou a cor 0x0000FF00) residem no mesmo espaço lógico de endereçamento da memória principal.
- **Entrada e Saída (E/S) Mapeada em Memória:** No modelo de Von Neumann, a UCP se comunica com os dispositivos de Entrada e Saída. O nosso TAP especifica a "Implementação de E/S mapeada em memória (Memory-Mapped I/O) para leitura do teclado e controle do Bitmap Display". Isso significa que o processador não usa instruções especiais externas para falar com o teclado ou a tela; em vez disso, ele lê e escreve em endereços de memória específicos (0xFFFF0000 e 0x10040000) como se fossem variáveis comuns, refletindo a filosofia de Von Neumann de um espaço de endereçamento unificado e compartilhado.
- Execução Sequencial, Ciclo de Instrução e o PC: O computador IAS funciona através de um ciclo contínuo de busca (fetch), decodificação e execução sequencial regido pelo IAR (Instruction Address Register). Em nosso projeto, o processador MIPS opera através do *Program Counter* (PC) rodando um "Laço de repetição principal (Game Loop) funcional". Este Game Loop é a representação em software do ciclo clássico: o processador busca continuamente as instruções, lê o estado do teclado via *polling*, calcula o movimento e atualiza a tela infinitamente, utilizando *Syscalls de sleep* para controlar a velocidade com que a UCP executa essa sequência no tempo.
- **Uso da Unidade de Lógica e Aritmética (ULA) e Registradores:** O TAP menciona explicitamente o "uso de registradores da UCP" como conceito fundamental. Enquanto o IAS dependia de um único Acumulador (AC) gerando

gargalos , o MIPS evolui utilizando um banco de 32 registradores de propósito geral (como `t0` e `a0`). Essa multiplicidade nos permite processar simultaneamente a lógica do jogo (movimentação, detecção de colisão) e manter o endereço base do Bitmap Display em cache, otimizando a renderização.

- **Arquitetura Load/Store:** Diferente do IAS, onde instruções aritméticas podiam interagir mais diretamente com a memória, o MIPS do nosso projeto é estritamente *Load/Store*. A relação com a memória ocorre exclusivamente via instruções `lw` (*Load Word*) e `sw` (*Store Word*). Isso força a ULA a operar apenas com dados que já estão nos registradores, essencial para injetar os pixels na tela e configurar as Entradas e Saídas.

4. Implementação Prática e Resultados Obtidos

Nesta etapa, o código-fonte base do jogo "Snake" foi totalmente estruturado, depurado e testado com sucesso no simulador MARS. O protótipo já atende aos requisitos iniciais de movimentação, leitura de teclado e renderização gráfica. Os principais marcos práticos alcançados incluem:

- **Estruturação de Dados na Memória:** A representação do corpo da cobra foi implementada utilizando **Arrays Paralelos** alocados na seção de dados (`.data`). Foram criados dois vetores contíguos (`cobra_col` e `cobra_lin`) que armazenam as coordenadas cartesianas de cada segmento da cobra. Essa abordagem otimizou o gerenciamento de memória e facilitou o acesso aos dados via registradores, evitando a complexidade do uso de matrizes bidimensionais.
- **Lógica de Deslocamento Otimizada:** O algoritmo de movimentação foi desenvolvido para minimizar ciclos de processamento. Em vez de recalcular o corpo inteiro a cada iteração do *Game Loop*, o sistema apenas desloca os valores dos vetores de trás para frente (onde o segmento `i` recebe a coordenada do segmento `i-1`) e, por fim, atualiza exclusivamente a coordenada da cabeça somando-a ao vetor da direção atual informada pelo teclado.
- **Geração de Elementos e Tratamento de Colisões:** Foram implementadas e

validadas as sub-rotinas de verificação de limites de tela (*Game Over* ao bater nas bordas) e de colisão da cabeça com o próprio corpo. Para a geração aleatória do objetivo (maçã), utilizou-se a *syscall 42 (Random Int Range)*. Durante a fase de testes, identificou-se um bug onde a maçã poderia ser gerada dentro da cobra; isso foi corrigido adicionando-se um laço de varredura que revalida as coordenadas sorteadas contra as posições atuais do vetor do corpo.

- **Depuração Crítica (Resolução de Conflito de Endereçamento):** Durante os testes de execução inicial, o sistema apresentou um erro de *Address out of range* seguido de falha crítica (crash). A equipe diagnosticou que o simulador estava utilizando o endereço base de vídeo configurado em `0x10010000`, o que causava uma sobreposição direta com a memória de dados estáticos (`.data`). Ao desenhar os pixels, o programa sobrescrevia os vetores de coordenadas do jogo. A solução arquitetural aplicada foi o realinhamento e isolamento do Bitmap Display, transferindo o endereço base de vídeo para a área de *Heap* (`0x10040000`), resultando em total estabilidade de I/O e renderização contínua.

Desenvolvimento do Jogo “Snake” em Linguagem Assembly

Gráfico de Gantt

Lista

16/03 24/03 30/03 08/04 14/04 22/04 30/04 01/05 08/05 25/05 10/06 22/06

Done

- Início do Projeto e Definição do Grupo
- Desenvolvimento do TAP
- Configuração do Ambiente MARS e Estrutura de Dados

On Progress

- Desenvolvimento do Motor Gráfico

On Going

- Implementação de E/S via Teclado
- Integração do Game Loop e Lógica de Movimento
- Testes e validações do Projeto
- Elaboração do Relatório Final Consolidado
- Defesa do Projeto

